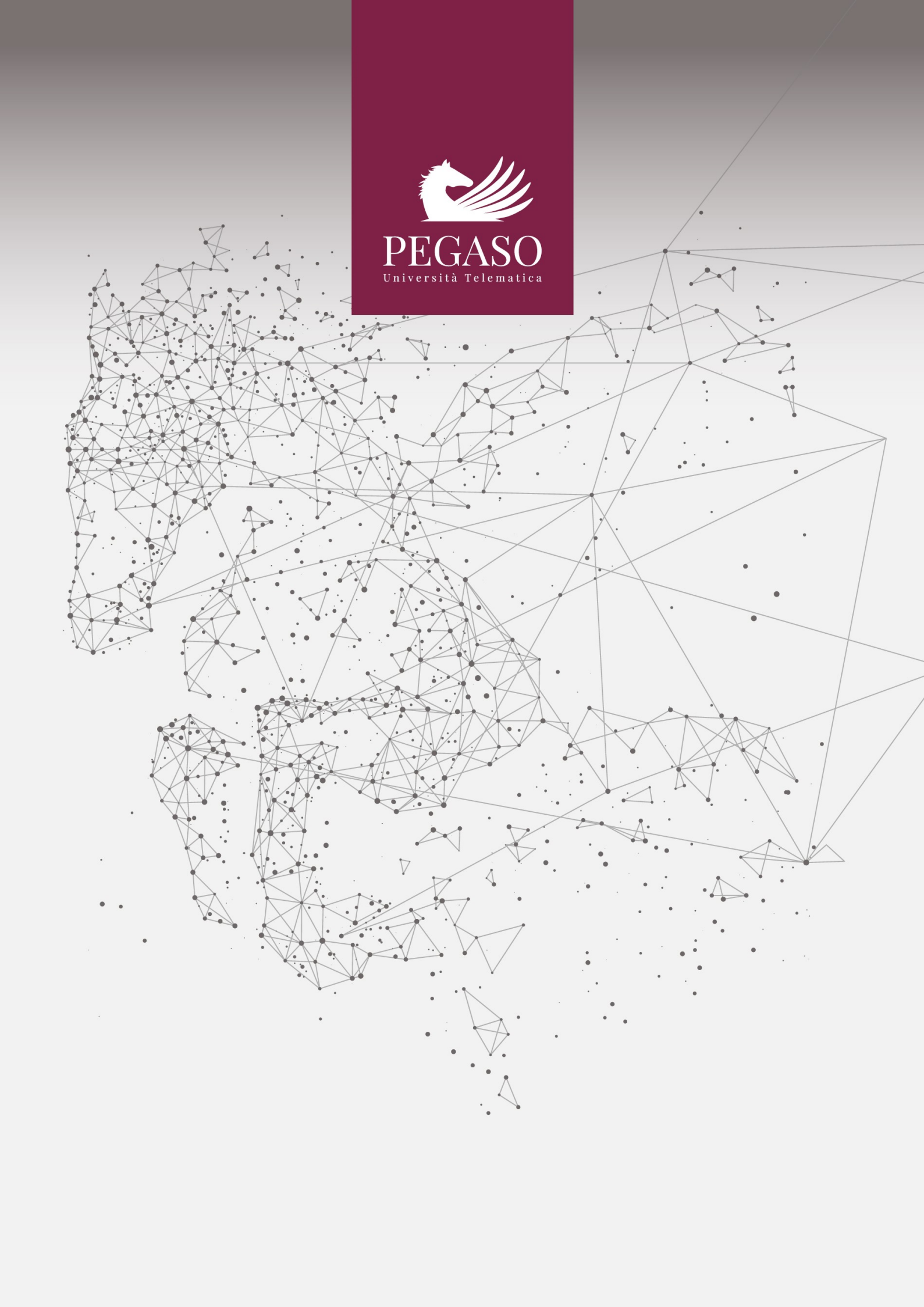

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. SUCCESSIONE DI FIBONACCI 4
3. ALGORITMI DI ORDINAMENTO 7
3.1 ALGORITMO BUBBLE SORT 7
3.2 ALGORITMO SELECTION SORT 12
4. ALGORITMI CON LISTE BIDIMENSIONALI 15
BIBLIOGRAFIA 28

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano una serie di esercizi che permettono di sviluppare le competenze di progettazione di algoritmi che operano direttamente sulle sequenze attraverso uno o più indici senza utilizzare i metodi già sviluppati in Python. Il primo esercizio illustrato è quello della successione di Fibonacci. Vengono, inoltre, illustrati due algoritmi di ordinamento delle sequenze. Infine, si approfondiscono proprietà e algoritmi delle liste bidimensionali.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Successione di Fibonacci

Anche se le liste offrono una serie di metodi per risolvere alcuni problemi tipici (es. ordinamento) è fondamentale saper progettare **algoritmi che operano direttamente sulle sequenze attraverso uno o più indici.** In questa lezione vedremo alcuni algoritmi che fanno **meno uso possibile dei metodi** messi a disposizione per le sequenze da Python.

Un esercizio tipico per lo sviluppo delle competenze necessarie è quello della nota **successione di Fibonacci.** L'intento originale di Leonardo Fibonacci era alquanto curioso: trovare una legge matematica che descrivesse la **crescita di una popolazione di conigli.** Le ipotesi alla base di questo problema sono 4:

1. si dispone di una coppia di conigli appena nati;
2. questa prima coppia diventa fertile al compimento del primo mese e dà alla luce una nuova coppia al compimento del secondo mese;
3. le nuove coppie nate si comportano in modo analogo;
4. le coppie fertili dal secondo mese di vita in poi danno alla luce una coppia di figli al mese.

Partendo dalle 4 ipotesi precedenti si verifica quanto segue:

1. dopo un mese, una coppia di conigli sarà fertile;
2. dopo due mesi, ci saranno due coppie di cui una sola fertile;
3. nel mese seguente, terzo mese dal momento iniziale, ci saranno 2+1=3 coppie perché solo la coppia fertile avrà generato; di queste tre, due saranno le coppie fertili;
4. nel mese seguente (quarto mese dal momento iniziale) ci saranno, quindi, 3+2=5 coppie;
5. il numero di coppie di conigli di ogni mese esprime la successione di Fibonacci.

Dal punto di vista matematico, la successione di Fibonacci **Fib(n)** indica una successione di numeri interi in cui ciascun numero è la somma dei due precedenti eccetto i primi due. I primi due, infatti, per definizione sono:

Fib(0) = 0

Fib(1) = 1

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La successione è definita ricorsivamente nel seguente modo:

Fib(n) = Fib(n-1) + Fib(n-2)

I primi termini della successione di Fibonacci sono:

0,1,1,2,3,5,8,13,24,34,55,89,144,233,…

La successione di Fibonacci ha un'interessante proprietà, ovvero che il limite per n tendente ad infinito del rapporto tra due numeri successivi della successione è pari alla sezione aurea:

$$\lim_{n\to\infty} \frac{Fib(n)}{Fib(n-1)} = \phi$$

dove ϕ è la sezione aurea o numero di Fidia pari a:

$$\phi = \frac{1+\sqrt{5}}{2} = 1{,}6180339887$$

Dal punto di vista informatico, l'esercizio è quello di raccogliere in input un valore intero n>2 e costruire una lista con i numeri della successione di Fibonacci fino a Fib(n).

Questo esercizio è importante dal punto di vista algoritmico perché presuppone **l'analisi e l'elaborazione non di un solo elemento per volta, ma di più elementi (in questo caso tre) per volta.** La definizione della formula di Fibonacci è infatti:

Fib(n) = Fib(n-1) + Fib(n-2)

Il codice sorgente che segue richiede in input il valore di n (con un ciclo di validazione dell'input, si assicura che n sia maggiore o uguale a 2). Si procederà, dunque, ad inizializzare la lista di **n+1 elementi** con i primi due elementi:

fib[0] = 0

fib[1] = 1

e si calcoleranno gli elementi dall'indice 2 all'indice n (per la funzione range indicheremo n+1 per poter raggiungere n) con la formula:

Fib(i) = Fib(i-1) + Fib(i-2)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    n=0
    while n<2:
        n = int(input("Inserire il valore di n per la formula \
                       di Fibonacci: (n>=2) "))
    fib = [0]*(n+1)
    fib[1] = 1

    for i in range(2,n+1):
        fib[i] = fib[i-1] + fib[i-2]

    print(fib)

main()
```

Il risultato dell'elaborazione del codice precedente è (con n inserito in input pari a 13):

```
Inserire il valore di n per la formula di Fibonacci: (n>=2) 13
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Algoritmi di ordinamento

Altri algoritmi che operano direttamente sulle sequenze attraverso uno o più indici sono gli algoritmi di ordinamento. In questa lezione analizzeremo due algoritmi di ordinamento:

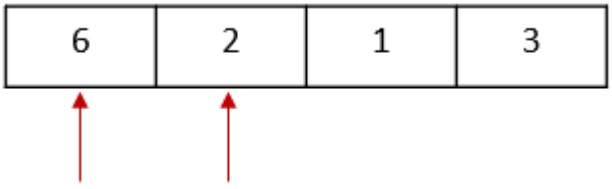
1) **bubble sort;**
2) **selection sort.**

3.1 Algoritmo bubble sort

L'algoritmo **bubble sort** (**ordinamento a bolla**) permette di ordinare i dati sia in ordine crescente che descrescente. Esso prevede che vengano **confrontate coppie di elementi adiacenti nella sequenza**. Se gli elementi della coppia analizzata sono nell'ordine errato (es. per ordinamento crescente elemento in posizione i maggiore dell'elemento in posizione i+1), gli elementi vengono invertiti di posizione.

Al termine dell'analisi di tutte le coppie, l'elemento maggiore/minore (secondo l'ordinamento crescente/decrescente) si troverà in ultima posizione nella sequenza provocando, quindi, l'effetto **bolla (bubble)**.

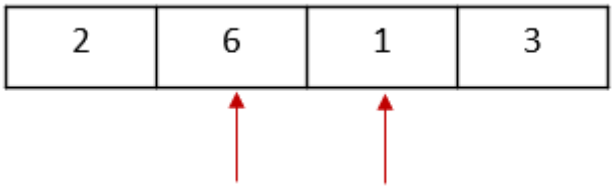
Si ipotizzi di voler ordinare una sequenza in ordine crescente. Nella sequenza di esempio che segue formata dai numeri 6,2,1,3 si confronta la prima coppia (primo e secondo elemento). Essendo 6 in posizione precedente al valore 2, si procede allo scambio dei due elementi:



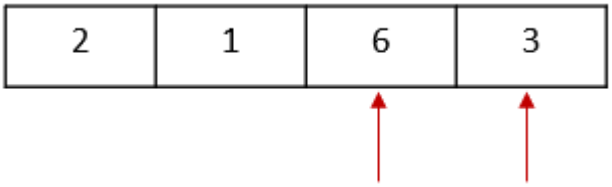
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

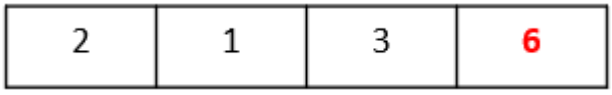
A seguito dello scambio di 6 e 2, si confronteranno gli elementi della seconda coppia consecutiva (secondo e terzo elemento). Anche in questo caso, 6 e 1 verranno invertiti in quanto 6 >1:



A seguito dello scambio di 6 e 1, si confronteranno gli elementi della terza e ultima coppia consecutiva (terzo e quarto elemento). Anche in questo caso, 6 e 3 verranno invertiti in quanto 6 >3:



A seguito dell'ultimo scambio, il 6 (che è il numero massimo presente nella sequenza) è finito in ultima posizione. Si è verificato, così, l'effetto bolla per cui il 6 è stato traslato lungo la sequenza verso destra.

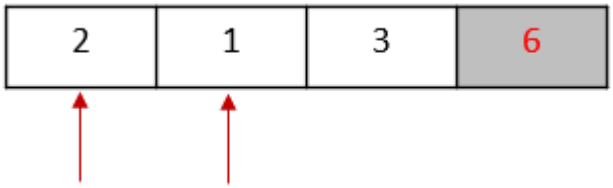


L'algoritmo bubble sort, dunque, ripete l'analisi della sequenza coppia per coppia dall'inizio, ma questa volta fino al penultimo elemento. Si confronta la prima coppia (primo e

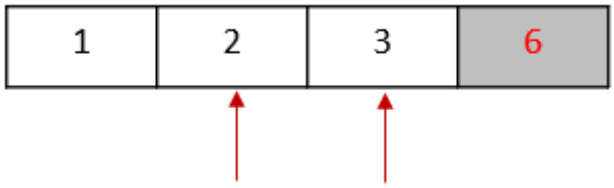
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

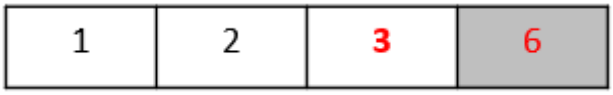
secondo elemento). Essendo 2 in posizione precedente al valore 1, si procede allo scambio dei due elementi:



A seguito dello scambio di 2 e 1, si confronteranno gli elementi della seconda coppia consecutiva (secondo e terzo elemento). Essendo 2 inferiore a 3, non si procederà con lo scambio:



In penultima posizione, avremo il numero maggiore della sottosequenza dal primo al penultimo elemento:



L'ultimo confronto verrà fatto tra il primo e il secondo elemento. Essendo già ordinati non si procederà con lo scambio.

Come visto nell'esempio precedente, l'algoritmo continua ogni volta non considerando gli elementi «bolla» già sistemati nella parte terminale della sequenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'algoritmo termina in una delle due condizioni seguenti:

1. durante l'analisi delle coppie (in una "passata" di analisi della sequenza) non avviene nessuno scambio;
2. tutti gli elementi della sequenza dal secondo all'ultimo sono stati considerati come elementi bolla.

Analizzeremo prima la versione semplificata del bubble sort in cui l'algoritmo termina solo quando tutti gli elementi della sequenza dal secondo all'ultimo sono stati considerati come elementi bolla.

Successivamente vedremo come ottimizzare l'algoritmo considerando come condizione di termine l'assenza di scambi durante l'analisi delle coppie in una «passata» di analisi della sequenza.

La funzione *bubble_sort*() riceve come parametro la lista di elementi da ordinare. Viene, quindi, assegnato ad n il numero degli elementi della lista.

Si procede impostando due cicli annidati:

1. il ciclo più esterno con indice i per impostare il numero di "passate" di analisi della sequenza pari a n-2;
2. il ciclo più interno per confrontare le coppie consecutive. Si noterà che l'indice j del ciclo più interno, al crescere di i si vedrà ridurre il limite superiore (n-i-1). Ciò avviene per tenere conto degli elementi bolla che sono stati sistemati nelle ultime posizioni della sequenza.

Nel ciclo più interno, si confrontano le coppie consecutive e si procede allo scambio degli elementi della sequenza se un elemento è maggiore dell'elemento che segue.

Vengono riportate due modalità di scambio degli elementi:

1. la prima utilizza una variabile/oggetto di appoggio per scambiare i due elementi (procedura seguita anche in linguaggi di programmazione come C++);
2. la seconda, commentata nel codice, utilizza una funzionalità di Python per cui a sinistra del simbolo = possono essere posti più elementi separati da virgola che prendono i valori corrispondenti degli elementi separati da virgola posti a destra del simbolo di uguale.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione *main*() crea una lista di elementi in ordine decrescente e la passa alla funzione bubble_sort() che provvede a restituire la lista ordina in modo crescente e a stamparla:

```
def bubble_sort(l):
    n = len(l)

    for i in range(n-1):
        for j in range(0, n-i-1):
            if l[j] > l[j + 1]:
                a = l[j]
                l[j] = l[j+1]
                l[j+1] = a
                #forma alternativa per lo scambio in Python
                #l[j], l[j + 1] = l[j + 1], l[j]
    return l

def main():
    l = list(range(10,0,-1))
    print("Lista originale:",l)
    l = bubble_sort(l)
    print("Lista ordinata in modo crescente:",l)


main()
```

Il risultato dell'elaborazione del programma precedente è:

```
Lista originale: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
Lista ordinata in modo crescente: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Procediamo, ora, con l'ottimizzazione dell'algoritmo considerando come condizione di terminazione l'assenza di scambi durante l'analisi delle coppie in una «passata» di analisi della sequenza. Per raggiungere tale obiettivo, utilizzeremo un flag denominato *scambio* che verrà inizializzato a False all'inizio di ogni "passata" di analisi della sequenza e segnalerà l'avvenuto/non avvenuto scambio di elementi in una coppia consecutiva. Il test del flag *scambio* verrà effettuato in and logico insieme al raggiungimento del penultimo elemento dell'indice i del ciclo più esterno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def bubble_sort(l):
    n = len(l)
    i = 0
    scambio = True
    while i<n-1 and scambio:
        scambio = False
        for j in range(0, n-i-1):
            if l[j] > l[j + 1]:
                a = l[j]
                l[j] = l[j+1]
                l[j+1] = a
                scambio = True
        i+=1
    return l

def main():
    l = [3,15,8,6,1,4,12,9]
    print("Lista originale:",l)
    l = bubble_sort(l)
    print("Lista ordinata in modo crescente:",l)


main()
```

Il risultato dell'elaborazione del programma precedente è:

```
Lista originale: [3, 15, 8, 6, 1, 4, 12, 9]
Lista ordinata in modo crescente: [1, 3, 4, 6, 8, 9, 12, 15]
```

3.2 Algoritmo selection sort

L'algoritmo **selection sort (ordinamento per selezione)** permette di ordinare i dati sia in ordine crescente che descrescente, ma opera in modo diverso rispetto al bubble sort. L'algoritmo, infatti, seleziona di volta in volta il numero minore nella sequenza di partenza e lo sposta nella sequenza ordinata. Di fatto la sequenza viene suddivisa in due parti:

- la sottosequenza ordinata, che occupa le prime posizioni dell'array;
- la sottosequenza da ordinare, che costituisce la parte restante dell'array.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Dovendo ordinare una sequenza **a** di lunghezza *n*, si fa scorrere l'indice i da 1 a n-1 ripetendo i seguenti passi:

1. si cerca il più piccolo elemento della sottosequenza a[i..n];
2. si scambia questo elemento con l'elemento i-esimo.

Nel codice sorgente che segue si definisce la funzione selection_sort() che riceve come parametro la lista da ordinare. Si copia in n il numero degli elementi della lista.

Si procede impostando due cicli annidati:

1. il ciclo più esterno con indice i per scorrere la sequenza;
2. il ciclo più interno con indice j per confrontare l'elemento di posizione i con tutti quelli presenti nella sottosequenza non ancora ordinata a destra della posizione i. Si noti, infatti, che il limite inferiore dell'intervallo dei valori dell'indice j è proprio i+1.

Per ogni iterazione del ciclo più esterno si inizializza l'indice del presunto numero più piccolo proprio ad i essendo la sottosequenza a sinistra di i già ordinata. Nel ciclo più interno, si confrontano gli elementi a destra della posizione di i per verificare se esiste un numero inferiore a quello in posizione i. In caso positivo, l'indice ind_min viene aggiornato al valore di j.

Al termine del ciclo più interno, se ind_min è stato aggiornato (quindi ind_min sarà diverso da i), avviene lo scambio dell'elemento in posizione i con l'elemento più piccolo della sottosequenza analizzata.

La funzione *main*() crea una lista di elementi casuali e la passa alla funzione selection_sort() che provvede a restituire la lista ordina in modo crescente e a stamparla.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def selection_sort(l):
    n = len(l)
    for i in range(n):
        ind_min = i
        for j in range(i+1, n):
            if l[ind_min] > l[j]:
                ind_min = j
        if i!=ind_min:
            l[i], l[ind_min] = l[ind_min], l[i]
        print(l)
    return l

def main():
    l = [3,15,8,6,1,4,12,9]
    print("Lista originale:",l)
    l = selection_sort(l)
    print("Lista ordinata in modo crescente:",l)


main()
```

Il risultato dell'elaborazione del programma precedente è:

```
Lista originale: [3, 15, 8, 6, 1, 4, 12, 9]
[1, 15, 8, 6, 3, 4, 12, 9]
[1, 3, 8, 6, 15, 4, 12, 9]
[1, 3, 4, 6, 15, 8, 12, 9]
[1, 3, 4, 6, 15, 8, 12, 9]
[1, 3, 4, 6, 8, 15, 12, 9]
[1, 3, 4, 6, 8, 9, 12, 15]
[1, 3, 4, 6, 8, 9, 12, 15]
[1, 3, 4, 6, 8, 9, 12, 15]
Lista ordinata in modo crescente: [1, 3, 4, 6, 8, 9, 12, 15]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Algoritmi con liste bidimensionali

Più complesse dal punto di vista della programmazione sono le **liste bidimensionali**, anche note come **matrici.** Una schematizzazione di una lista bidimensionale può essere la seguente:

		Indice di colonna		
		0	1	2
Indice di riga	0	lista2D[0,0]	lista2D[0,1]	lista2D[0,2]
	1	lista2D[1,0]	lista2D[1,1]	lista2D[1,2]
	2	lista2D[2,0]	lista2D[2,1]	lista2D[2,2]
	3	lista2D[3,0]	lista2D[3,1]	lista2D[3,2]

Un esercizio molto interessante è quello del **calcolo della matrice trasposta** di una matrice di partenza. La matrice trasposta di una matrice è la **matrice ottenuta scambiandone le righe con le colonne.**

Gli elementi della matrice di partenza m[i][j] saranno, dunque, uguali agli elementi mt[j][i] della trasposta, come indicato nell'esempio seguente:

$$A = \begin{pmatrix} 2 & 4 & 8 \\ 3 & 2 & 0 \\ 5 & 3 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad A^T = \begin{pmatrix} 2 & 3 & 5 & 0 \\ 4 & 2 & 3 & 1 \\ 8 & 0 & 1 & 0 \end{pmatrix}$$

Nel codice sorgente che segue, attraverso la funzione inizializza(), si inizializza una lista bidimensionale di 3 righe e 2 colonne con numeri random. In dettaglio, prima viene costruita, attraverso l'operatore di ripetizione, una matrice di elementi nulli. Successivamente, richiamando la funzione randint() del modulo random, si assegnano valori casuali agli elementi della matrice indicizzati con i e j.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione restituisce la matrice risultante:

```
import random

RIGHE = 3
COLONNE = 2

def inizializza():
    m = [0]*RIGHE
    for i in range(RIGHE):
        riga = [0]*COLONNE
        m[i] = riga

    for i in range(RIGHE):
        for j in range(COLONNE):
            m[i][j] = random.randint(1,9)

    return m
```

La funzione trasposta() restituisce la matrice trasposta della matrice passata come argomento. In primo luogo, la funzione trasposta() inizializza una matrice con il numero di righe della matrice originale invertito rispetto al numero di colonne della stessa e viceversa.

Si procede, dunque, con la copia degli elementi della matrice originale nella matrice trasposta con gli indici invertiti (indice di riga diventa indice di colonna nella trasposta, indice di colonna diventa indice di riga nella trasposta).

```
def trasposta(m):

    mt = [0]*COLONNE
    for i in range(COLONNE):
        riga = [0]*RIGHE
        mt[i] = riga

    for i in range(RIGHE):
        for j in range(COLONNE):
            mt[j][i] = m[i][j]

    return mt
```

La funzione main(), inizializza la matrice di partenza e la visualizza a schermo.

Successivamente, richiama la funzione trasposta() a cui passa la matrice di partenza. Il risultato della funzione viene assegnato ad m2 che viene, quindi, visualizzata a schermo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():

    m = inizializza()

    print("Matrice:")
    for i in range(RIGHE):
        print(m[i])

    m2 = trasposta(m)

    print()
    print("Matrice trasposta:")
    for i in range(COLONNE):
        print(m2[i])


main()
```

Il risultato dell'elaborazione del programma precedente è:

```
Matrice:
[4, 5]
[6, 3]
[8, 4]

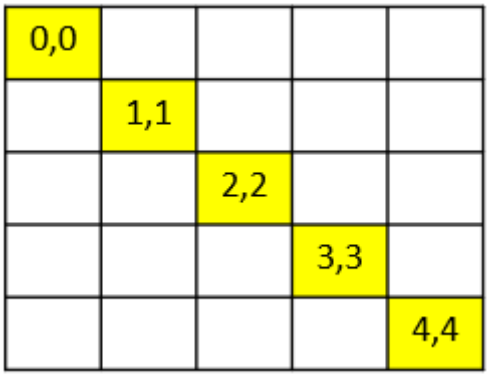
Matrice trasposta:
[4, 6, 8]
[5, 3, 4]
```

Le matrici che hanno stesso numero di righe e colonne sono dette **matrici quadrate.** Le matrici quadrate hanno interessanti proprietà e caratteristiche. Ad esempio, è possibile definire la **diagonale principale** della matrice quadrata. La diagonale principale è la diagonale che va dall'angolo in alto a sinistra a quello in basso a destra.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La caratteristica degli elementi presenti sulla diagonale principale è che l'indice di riga è uguale a quello di colonna, come visibile nella matrice seguente:



Si propone, ora, un esercizio per cui in una matrice 5x5 con tutti gli elementi nulli, si vogliono porre gli elementi della diagonale principale ad 1. Inizializzeremo la matrice con tutti gli elementi a zero e, successivamente, con un solo ciclo for di 5 iterazioni andremo a sovrascrivere i 5 elementi della diagonale principale.

Dichiariamo una variabile globale N che poniamo uguale a 5 (pari al numero di righe che è uguale al numero di colonne della matrice). Dichiariamo, quindi, la funzione inizializza() che restituisce una matrice di dimensioni NxN di valori tutti nulli.

Con la funzione diag_princip() sovrascriviamo tutti gli elementi con indice di riga uguale all'indice di colonna (gli elementi della diagonale principale) ponendoli uguali ad 1.

La funzione visualizza_mat() permetterà di visualizzare la matrice in una modalità più comprensibile rispetto alla semplice funzione print().

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
N = 5

def inizializza():
    m = [0]*N
    for i in range(N):
        riga = [0]*N
        m[i] = riga
    return m


def diag_princip(m):
    for i in range(N):
        m[i][i] = 1
    return m


def visualizza_mat(m):
    for i in range(N):
        print(m[i])
```

La funzione main(), dunque, richiamerà la funzione inizializza() per ottenere la matrice m. In seguito, richiamerà la funzione visualizza_mat() passandogli la matrice m. Procederà richiamando la funzione diag_princip() il cui risultato verrà sovrascritto in m. Infine, verrà visualizzata a schermo la matrice con gli elementi della diagonale principale tutti uguali ad 1.

```
def main():
    m = inizializza()
    print("Matrice:")
    visualizza_mat(m)

    m = diag_princip(m)

    print()
    print("Matrice con diagonale principale a 1:")
    visualizza_mat(m)


main()
```

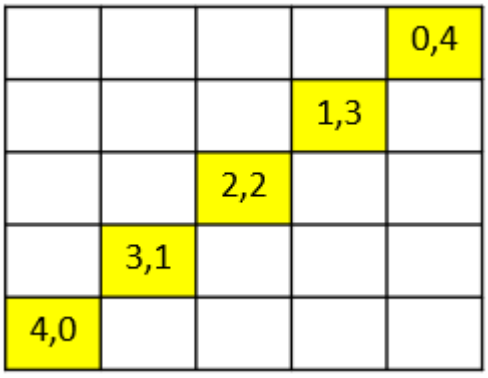
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione del programma precedente è:

```
Matrice:
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]

Matrice con diagonale principale a 1:
[1, 0, 0, 0, 0]
[0, 1, 0, 0, 0]
[0, 0, 1, 0, 0]
[0, 0, 0, 1, 0]
[0, 0, 0, 0, 1]
```

È possibile definire anche la **diagonale secondaria**, che è la **diagonale** che va dall'angolo in alto a destra a quello in basso a sinistra. Per poter trovare la relazione tra gli indici, è utile considerare che la somma degli indici di tali elementi è sempre uguale a N-1 come è possibile osservare nella seguente matrice.



Segue che:

$$i+j = N-1$$

quindi:

$$j(i) = N-1-i$$

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice sorgente che segue utilizza le stesse funzioni dichiarate per il caso della diagonale principale, ad eccezione della funzione diag_second() che provvede a sovrascrivere con un solo ciclo for di N iterazioni gli elementi della diagonale secondaria della matrice passata come parametro.

```
N = 5

def inizializza():
    m = [0]*N
    for i in range(N):
        riga = [0]*N
        m[i] = riga
    return m


def diag_second(m):
    for i in range(N):
        m[i][N-1-i] = 1
    return m


def visualizza_mat(m):
    for i in range(N):
        print(m[i])


def main():
    m = inizializza()
    print("Matrice:")
    visualizza_mat(m)

    m = diag_second(m)

    print()
    print("Matrice con diagonale secondaria a 1:")
    visualizza_mat(m)


main()
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione del programma precedente è:

```
Matrice:
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]

Matrice con diagonale secondaria a 1:
[0, 0, 0, 0, 1]
[0, 0, 0, 1, 0]
[0, 0, 1, 0, 0]
[0, 1, 0, 0, 0]
[1, 0, 0, 0, 0]
```

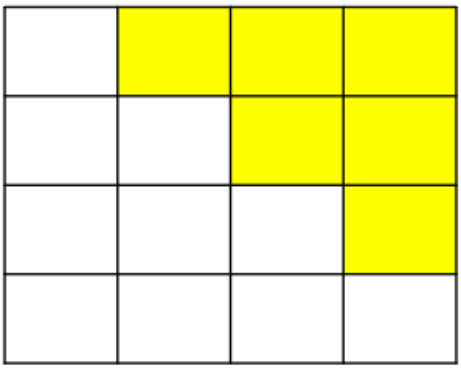
Per le matrici quadrate è possibile verificare se esse sono **simmetriche rispetto alla diagonale principale.** Una matrice simmetrica è una matrice quadrata che ha la proprietà di essere la **trasposta di sé stessa.** In altre parole, gli elementi m[i][j] sono uguali a m[j][i] come è possibile osservare nella matrice seguente:

3	2	1
2	4	0
1	0	5

È importante sottolineare che per non eseguire confronti ridondanti, i confronti devono essere fatti considerando la seguente porzione di matrice evidenziata in giallo (in caso contrario, i controlli sarebbero doppi).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Iterando con un ciclo esterno sull'indice di riga i, è semplice verificare che l'indice di colonna j del ciclo interno dovrà partire da i+1 e arrivare a N-1 per ogni riga.

Definiamo, quindi, una funzione is_simmetrica() che restituisce True se la matrice passata come argomento è simmetrica, False in caso contrario. La funzione, quindi, controlla che gli elementi di indice [i][j] siano uguali a quelli di indice [j][i]. Non appena il programma incontra un caso per cui questa uguaglianza non è verificata, la funzione termina restituendo un valore False. Se i cicli annidati terminano senza mai verificare la condizione dell'istruzione if, vuol dire che la matrice è simmetrica, quindi la funzione restituirà valore True.

```
N = 5

def inizializza():
    m = [0]*N
    for i in range(N):
        riga = [0]*N
        m[i] = riga
    return m


def is_simmetrica(m):
    for i in range(N):
        for j in range(i+1,N):
            if m[i][j] != m[j][i]:
                return False
    return True


def visualizza_mat(m):
    for i in range(N):
        print(m[i])
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def main():
    m = inizializza()
    print("Matrice:")
    visualizza_mat(m)

    print()
    if is_simmetrica(m):
        print("Matrice simmetrica")
    else:
        print("Matrice non simmetrica")


main()
```

Per una matrice di elementi tutti nulli, il risultato dell'elaborazione del programma precedente è:

```
Matrice:
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]
[0, 0, 0, 0, 0]

Matrice simmetrica
```

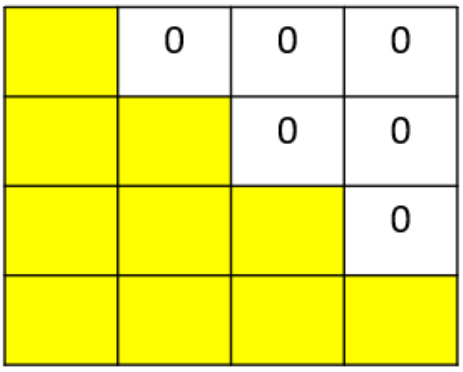
Partendo dal caso precedente, andiamo a definire la **matrice triangolare superiore.** In queste matrici, gli elementi «al di sotto della diagonale principale» sono tutti nulli, mentre quelli nelle posizioni evidenziate in giallo sono non nulli.

0			
0	0		
0	0	0	

Nella **matrice triangolare inferiore** avviene il contrario, cioè gli elementi «al di sopra della diagonale principale» sono tutti nulli, mentre quelli nelle posizioni evidenziate in giallo sono non nulli.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Partendo da matrici NxN ottenute mediante la funzione inizializza(), andiamo a definire due funzioni: matr_triang_sup() per inizializzare una matrice triangolare superiore con elementi casuali e matr_triang_inf() per inizializzare una matrice triangolare inferiore con elementi casuali.

In particolare, alla matrice NxN di elementi tutti nulli:

- la funzione matr_traing_sup() sovrascrive tutti gli elementi della diagonale principale e "al di sopra della diagonale principale" con elementi casuali restituiti dalla funzione random.randint(). In questo caso è possibile osservare che l'intervallo di valori assunti dall'indice di riga i è [0, N-1], mentre l'intervallo di valori assunti dall'indice di colonna j è [i, N-1];
- la funzione matr_traing_inf() sovrascrive tutti gli elementi della diagonale principale e "al di sotto della diagonale principale" con elementi casuali restituiti dalla funzione random.randint(). In questo caso è possibile osservare che l'intervallo di valori assunti dall'indice di riga i è [0, N-1], mentre l'intervallo di valori assunti dall'indice di colonna j è [0,i].

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import random
N = 5

def inizializza():
    m = [0]*N
    for i in range(N):
        riga = [0]*N
        m[i] = riga
    return m


def matr_triang_sup(m):
    m2 = m
    for i in range(N):
        for j in range(i,N):
            m[i][j] = random.randint(0,9)
    return m2

def matr_triang_inf(m):
    m2 = m
    for i in range(N):
        for j in range(i+1):
            m[i][j] = random.randint(0,9)
    return m2

def visualizza_mat(m):
    for i in range(N):
        print(m[i])


def main():
    m = inizializza()
    print("Matrice triangolare superiore:")
    visualizza_mat(matr_triang_sup(m))

    print()
    m = inizializza()
    print("Matrice triangolare inferiore:")
    visualizza_mat(matr_triang_inf(m))

main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione del programma precedente è:

```
Matrice triangolare superiore:
[0, 2, 8, 7, 0]
[0, 1, 9, 9, 9]
[0, 0, 1, 5, 9]
[0, 0, 0, 4, 7]
[0, 0, 0, 0, 3]

Matrice triangolare inferiore:
[8, 0, 0, 0, 0]
[6, 5, 0, 0, 0]
[2, 3, 2, 0, 0]
[0, 9, 9, 1, 0]
[3, 4, 4, 6, 5]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).